PasswordStore Audit Report

Abdulmaleek

April 22, 2026

Contents

PasswordStore Audit Report 2

About the Auditor 2

Disclaimer 2

Risk Classification 2

Audit Details 3
- Scope . . . 3

Protocol Summary 3
- Roles . . . 3

Executive Summary 4
- Issues Found . . . 4

Findings 4
- High . . . 4
  - [H-1] Missing Access Control on `setPassword()` – Anyone Can Overwrite the Password . . . 4
  - [H-2] Passwords Stored On-Chain Are Visible to Anyone . . . 5
- Low . . . 6
  - [L-1] Incorrect NatSpec on `getPassword()` . . . 6
- Informational . . . 7
  - [I-1] `s_owner` Should Be Declared `immutable` . . . 7
  - [I-2] Solidity `0.8.18` Has Known Compiler Issues . . . 7

Conclusion 8

PasswordStore Audit Report

Prepared by: Abdulmaleek
Lead Auditor: Abdulmaleek
Assisting Auditors: None
Date: April 22, 2026
Version: 1.0

About the Auditor

This audit was conducted as part of the Cyfrin Updraft smart contract security course. The auditor reviewed the PasswordStore contract for security vulnerabilities, logic errors, and best practice violations.

Disclaimer

The auditor makes every effort to find as many vulnerabilities as possible within the given time period, but holds no responsibility for the findings provided in this document. A security audit is not an endorsement of the underlying business or product. The audit was time-boxed and the review was solely focused on the security aspects of the Solidity implementation.

Risk Classification

		Impact: High	Impact: Medium	Impact: Low
Likelihood	High	Critical	High	Medium

	Impact: High	Impact: Medium	Impact: Low
Medium	High	Medium	Low
Low	Medium	Low	Info

Audit Details

The findings described in this document correspond to the following commit hash:

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
src/
--- PasswordStore.sol
```

Protocol Summary

PasswordStore is a protocol dedicated to the storage and retrieval of a user's password. The protocol is designed to be used by a single user, and is not intended for multiple users. Only the owner should be able to set and access this password.

Roles

- **Owner**: The only account that should be able to set and access the password.

Executive Summary

Issues Found

Severity	Number of Issues Found
High	2
Medium	0
Low	1
Info	2
Total	**5**

Findings

High

[H-1] Missing Access Control on `setPassword()` – Anyone Can Overwrite the Password

Description

The `PasswordStore::setPassword` function is declared `external` with no access control. The NatSpec and overall purpose of the contract state that only the owner should be able to set a new password, but this is never enforced in code.

```
function setPassword(string memory newPassword) external {
    // @audit - No access control check here
    s_password = newPassword;
    emit SetNewPassword();
}
```

Impact

Any user or attacker can call `setPassword()` and permanently overwrite the owner's stored password. The owner loses all control of the contract's core functionality.

Proof of Concept

Add the following to the `PasswordStore.t.sol` test suite:

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);
    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation

Add an access control check using the existing error:

```
function setPassword(string memory newPassword) external {
    if (msg.sender != s_owner) {
        revert PasswordStore__NotOwner();
    }
    s_password = newPassword;
    emit SetNewPassword();
}
```

[H-2] Passwords Stored On-Chain Are Visible to Anyone

Description

All data stored on-chain is publicly readable, regardless of Solidity variable visibility. The `s_password` variable is declared **`private`**, but **`private`** only prevents other contracts from reading it – it does not hide data from anyone inspecting the blockchain directly.

Impact

The password is not private. Anyone can read it directly from blockchain storage, completely defeating the purpose of the contract.

Proof of Concept

1. Start a local chain:

```
make anvil
```

2. Deploy the contract:

```
make deploy
```

3. Read the storage slot directly (slot 1 = s_password):

```
cast storage <CONTRACT_ADDRESS> 1 --rpc-url http://127.0.0.1:8545
```

Output:

```
0x6d7950617373776f72640000000000000000000000000000000000000000000014
```

4. Decode the hex to a string:

```
cast parse-bytes32-string 0
    x6d7950617373776f72640000000000000000000000000000000000000000000014
```

Output:

```
myPassword
```

Recommended Mitigation

The overall architecture of the contract should be reconsidered. Encrypt the password off-chain and store only the encrypted value on-chain. Also remove the `getPassword()` view function to prevent accidental on-chain exposure of decryption keys.

Low

[L-1] Incorrect NatSpec on `getPassword()`

Description

The `getPassword()` function contains a copy-paste error in its NatSpec. It references a non-existent `@param newPassword` that has no relation to a getter function.

Impact

Misleading documentation reduces code auditability and can confuse developers integrating with the contract.

Recommended Mitigation

Remove the incorrect `@param` line:

```
/*
 * @notice This allows only the owner to retrieve the password.
 */
function getPassword() external view returns (string memory) {
```

Informational

[I-1] `s_owner` Should Be Declared `immutable`

Description

The `s_owner` variable is set once in the constructor and never modified. Declaring it `immutable` saves gas and signals intent clearly to readers.

Recommended Mitigation

```
address private immutable s_owner;
```

[I-2] Solidity `0.8.18` Has Known Compiler Issues

Description

Slither flagged that Solidity `0.8.18` has some known compiler bugs. None are critical for this specific contract, but it is best practice to use a recent stable version.

Recommended Mitigation

Consider upgrading to `^0.8.24` or the latest stable Solidity release.

Conclusion

This contract serves as an excellent learning example but is **not production-ready** in its current state.

Priority: Resolve H-1 (missing access control) and H-2 (on-chain password visibility) before any deployment.

This audit reinforces a fundamental rule of smart contract security:

> *Never trust the comments. Always verify the actual code.*